Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio

BugSlayer

Integrantes:
Daniel Alejandro Estupiñán Reyes (C-311) Pregunta 1
René Díaz González (C-311) Pregunta 2
Gilberto Rodríguez Crespo (C-311) Pregunta 3
David Castillo Rodríguez (C-311) Pregunta 4
Liz Cartaya Salabarría (C-311) Pregunta 5
Maya Ramón Castellanos (C-312) Pregunta 6

13 de noviembre de 2026
`Enlace al Repositorio en GitHub:`
`https://github.com/cartayaliz/travel-itinerary-generator.git`

1. Análisis Comparativo de Comunicación y Acoplamiento

Un sistema de subastas en línea requiere que, al finalizar una subasta, se realicen múltiples acciones: notificar al ganador, procesar el pago y actualizar el inventario. En un diseño con Microservicios, un arquitecto podría optar por una coreografía de eventos (comunicación asíncrona) o una orquestación mediante un servicio mediador. En un diseño con Service-Based Architecture, estas acciones probablemente se ejecutarían dentro de un mismo servicio de dominio de grano grueso. Basado en los conceptos de connascence (connascencia) y los trade-offs de la comunicación síncrona vs. asíncrona, analice:

Punto de partida:

Richards y Ford (2020) distinguen entre connascencia estática, que se ve leyendo el código (nombres, tipos, posiciones), y connascencia dinámica, que solo aparece en ejecución: de ejecución (el orden importa), de temporización, de valores y de identidad. Ford et al. (2021) llevan esto a los sistemas distribuidos y plantean que el acoplamiento dinámico entre servicios depende de tres decisiones: comunicación (síncrona o asíncrona), consistencia (atómica o eventual) y coordinación (orquestación o coreografía). Usaremos esas tres dimensiones para comparar los diseños. En la subasta, al cerrar se deben hacer tres cosas: notificar al ganador, procesar el pago y actualizar el inventario.

1.1. ¿Cómo afecta cada enfoque (orquestación vs. coreografía en microservicios, y servicios de grano grueso en SBA) al acoplamiento dinámico entre los componentes del sistema?

Coreografía en microservicios. El servicio de Subastas publica un evento SubastaFinalizada y los servicios de Notificaciones, Pagos e Inventario reaccionan por su cuenta. Ninguno conoce a los demás ni espera por ellos, así que desaparecen casi por completo la connascencia de ejecución y de temporización entre servicios. A cambio, el flujo queda implícito: el orden real de los pasos no está escrito en ningún lugar, sino repartido en las suscripciones. Además queda un acoplamiento sutil por el contrato del evento (connascencia de tipo y de valores): si cambia el formato del monto, todos los consumidores se rompen (Ford et al., 2021).

Orquestación en microservicios. Un servicio mediador (por ejemplo, OrquestadorCierreSubasta) invoca a cada servicio en el orden definido. La connascencia de ejecución no se elimina, se concentra en un solo punto. Los servicios quedan desacoplados entre sí, pero el orquestador depende de todos, y si usa llamadas síncronas aparece además connascencia de temporización: debe esperar la respuesta de cada uno. Se gana claridad y control del flujo, y se pierde autonomía y se crea un punto único de acoplamiento (Richards & Ford, 2020, cap. 17; Ford et al., 2021).

Service-Based Architecture (SBA). Las tres acciones ocurren dentro de un mismo servicio de dominio de grano grueso, como llamadas locales. El acoplamiento dinámico entre servicios distribuidos prácticamente no existe, porque no hay red entre los pasos. El acoplamiento no desaparece: se traslada al interior del servicio, donde la connascencia de ejecución y de valores se maneja con código y no con contratos. Si el servicio está mal modularizado internamente, ese acoplamiento se vuelve difícil de ver y de cambiar (Richards & Ford, 2020, cap. 13).

Aspecto	**Coreografía**	**Orquestación**	**SBA (grano grueso)**
Comunicación típica	Asíncrona	Síncrona o asíncrona	Local (en proceso)
Dónde vive el orden del flujo	Implícito, distribuido	Explícito, en el mediador	Explícito, en el código del servicio
Acoplamiento entre servicios	Bajo (vía contrato de evento)	Medio (orquestador-servicios)	Mínimo (todo adentro)
Visibilidad del flujo	Baja	Alta	Alta

1.2. ¿Qué implicaciones tiene cada diseño para la tolerancia a fallos y la consistencia de los datos (ACID vs. BASE) si el servicio de pago falla justo después de que se notifica al ganador?

Coreografía. El servicio de Notificaciones ya reaccionó al evento original, así que el ganador recibió su aviso aunque el pago no se haya completado. El sistema es eventualmente consistente (modelo BASE): hay una ventana en que los estados no coinciden. Para corregirlo, Pagos debe publicar PagoFallido y los demás servicios deben compensar: avisar al usuario, liberar el inventario y quizás ofrecer el lote al segundo mejor postor. Esto es una saga coreografiada (Garcia-Molina & Salem, 1987; Ford et al., 2021). Lo bueno es que la falla de un servicio no detiene a los demás. Lo malo es que la compensación es difícil de rastrear y de probar, porque no hay un lugar donde se vea el estado global.

Orquestación. El mediador conoce el estado del proceso y puede decidir el orden, por ejemplo cobrar primero y notificar solo si el pago se confirma, con lo que el problema del aviso prematuro ni siquiera se presenta. Si el pago falla, el orquestador ejecuta las compensaciones de forma explícita (saga orquestada). La consistencia sigue siendo eventual y no ACID, pero es más controlable y fácil de auditar. El costo es que el orquestador pasa a ser un cuello de botella y un punto único de falla, y si está caído, ninguna subasta cierra.

SBA. Aquí la consistencia fuerte es posible: los tres pasos pueden ejecutarse en una transacción ACID sobre la base de datos compartida, y si el pago falla se revierte todo (Richards & Ford, 2020, cap. 13). Hay un matiz importante: el aviso al ganador (un correo, una notificación push) es un efecto externo que no se puede deshacer con un rollback. Por eso, incluso en SBA, lo correcto es notificar después de confirmar la transacción. Y si el pago depende de una pasarela externa, tampoco queda dentro de la transacción local y se necesita compensación. El precio de este enfoque es un mayor radio de impacto: un error en el cierre de subastas afecta a todo el servicio, y este escala y se despliega como una sola unidad.

Conclusión

No hay un diseño que gane en todo. Si la prioridad es la consistencia y el equipo es pequeño, SBA es la opción más segura. Si se necesita escalar y desplegar cada capacidad por separado, los microservicios son mejores, y entre ambas variantes la orquestación da más control sobre los fallos y la coreografía más autonomía a cambio de menos visibilidad. En cualquier caso, el escenario revela un error de diseño común: notificar antes de confirmar el pago. Mover la notificación al final del flujo reduce el daño con independencia del estilo elegido.

2. Diseño para Elasticidad Extrema y Consistencia de Datos

Considere el sistema de venta de entradas para conciertos descrito en el libro (Capítulo 15). Durante el lanzamiento de entradas para un artista popular, el sistema experimenta un aumento repentino de usuarios de cientos a decenas de miles en segundos. Se le pide como arquitecto diseñar la solución.

2.1. Justifique por qué una Space-Based Architecture sería una elección más adecuada que una Microservices Architecture para este escenario específico, enfocándose en las características de elasticidad y escalabilidad.

Para un escenario de venta masiva de entradas con picos repentinos e impredecibles de cientos a decenas de miles de usuarios concurrentes en cuestión de segundos, la Space-Based Architecture (SBA) ofrece ventajas estructurales superiores a la Microservices Architecture (MSA) tradicional debido a la forma en que aborda el cuello de botella central de la persistencia de datos y la elasticidad inmediata (Richards & Ford, 2020, cap. 15):

1.Eliminación del Cuello de Botella en la Base de Datos:

Microservices Architecture: En un diseño típico de microservicios, cada servicio maneja su propia base de datos para preservar la autonomía y el desacoplamiento (Richards & Ford, 2020, cap. 17). Sin embargo, bajo una demanda concentrada y simultánea durante la salida a la venta de entradas, miles de solicitudes concurrentes compiten por leer y modificar la disponibilidad de los mismos asientos numerados. Esta concurrencia genera un bloqueo masivo y contención de transacciones en la capa de persistencia. Incluso aplicando técnicas de particionamiento, los hotspots de los asientos más codiciados saturan inevitablemente los motores de bases de datos relacionales o NoSQL tradicionales (Ford et al., 2021, cap. 4).

Space-Based Architecture: Diseñada específicamente para resolver los problemas de alta concurrencia y elasticidad causada por la persistencia centralizada, la SBA elimina la base de datos de la ruta crítica de las transacciones (Richards & Ford, 2020, cap. 15). El sistema opera mediante Unidades de Procesamiento autónomas que procesan las solicitudes directamente contra una red de datos en memoria. Al evitar el acceso síncrono a disco o motores de base de datos durante el pico de carga, la latencia se reduce drásticamente a microsegundos.

2.Elasticidad Extrema y Auto-escalado Dinámico:

La SBA incorpora componentes clave como el Deployment Manager y el Virtual Middleware (Richards & Ford, 2020, cap. 15). Ante un incremento abrupto en el volumen de usuarios, el Deployment Manager detecta la carga e instancia dinámicamente nuevas Unidades de Procesamiento con sus respectivas réplicas de datos en memoria. En el escenario del sistema de boletaje, el Deployment Manager puede pre-desplegar instancias adicionales momentos antes del inicio de la venta para mantenerlas en standby, absorbiendo el pico sin degradar la respuesta del sistema.

2.2. Sin embargo, la Space-Based Architecture introduce el problema de las colisiones de datos en cachés replicadas. Describa cómo un arquitecto podría mitigar el riesgo de colisiones de datos en el contexto de la venta de asientos numerados, donde la consistencia es crucial. ¿Qué mecanismos o trade-offs consideraría?

En una SBA, la replicación de datos en memoria entre múltiples nodos introduce el riesgo de colisiones de datos o condiciones de carrera: dos usuarios conectados a distintas unidades de procesamiento podrían intentar seleccionar y comprar simultáneamente el mismo asiento numerado (Richards & Ford, 2020, cap. 15). Para mitigar este riesgo garantizando la consistencia exacta en asientos numerados, un arquitecto debe implementar las siguientes técnicas y mecanismos:

1.Particionamiento y Enrutamiento Guiado por Datos (Data-Driven Routing):

En lugar de replicar el inventario global de asientos en todos los nodos, la cuadrícula de datos en memoria se subdivide o particiona por zonas, sectores o filas del evento (Richards & Ford, 2020, cap. 15). Mediante un componente de enrutamiento, cada solicitud de reserva para un asiento específico se encamina de forma unívoca a la única unidad de procesamiento propietaria de esa partición. Al centralizar la autoridad de modificación de un asiento en una sola unidad lógica en memoria, se eliminan las escrituras concurrentes desde múltiples nodos y se evitan las colisiones sin recurrir a bloqueos distribuidos costosos.

2. Control de Concurrencia Optimista con Versionado:

Si se permite la lectura replicada del mapa de asientos en múltiples nodos, las operaciones de actualización utilizan un mecanismo de versionado. Cuando un usuario intenta reservar el asiento versión V1, el sistema verifica atómicamente si la versión en el grid sigue siendo V1. Si otro proceso modificó el asiento a V2, la transacción se rechaza de inmediato y se notifica al usuario que el asiento ya no está disponible (Ford et al., 2021, cap. 6).

3.Patrón de Bloqueo Temporal:

Al seleccionar un asiento, la unidad de procesamiento concede un bloqueo por tiempo determinado de 5 a 10 minutos en el grid en memoria. Si el usuario no completa la pasarela de pago dentro del tiempo

límite, el estado del asiento se libera automáticamente para otros compradores.

Trade-offs Arquitectónicos:

- Consistencia Estricta vs. Escalabilidad Lineal: Garantizar la consistencia absoluta para evitar reservas dobles exige que el acceso a un asiento numerado se canalice por una única partición propietaria. Esto introduce un trade-off: se limita la capacidad de escalar horizontalmente el procesamiento de un único asiento muy codiciado a cambio de evitar datos inconsistentes (Ford et al., 2021, cap. 4).

- Complejidad de Sincronización Asíncrona: La persistencia final hacia la base de datos centralizada se realiza de forma asíncrona mediante componentes como el Data Writer o Asynchronous Persister (Richards & Ford, 2020, cap. 15). Esto implica aceptar consistencia eventual a nivel de la base de datos física, asumiendo la complejidad de manejar fallos si un nodo en memoria cae antes de sincronizar sus cambios con el almacenamiento persistente.

3. Análisis de Fallos y Resiliencia en Arquitecturas Dirigidas por Eventos

En una Event-Driven Architecture con topología de *Broker*, un procesador de eventos (*Event Processor*) falla repetidamente al procesar un tipo específico de mensaje debido a un error en los datos.

3.1. Describa el impacto de este fallo en el flujo general del sistema y en la experiencia del usuario final, considerando que no hay un mediador central.

En una Arquitectura Dirigida por Eventos (EDA) que utiliza una topología de Broker, no existe un componente mediador centralizado que supervise, ordene o gestione reintentos en el flujo de ejecución; los procesadores reaccionan de manera autónoma e independiente ante los eventos publicados (Richards & Ford, 2020, cap. 14). Si un procesador de eventos (Event Processor) falla repetidamente al procesar un mensaje específico, el impacto se manifiesta en dos dimensiones clave:

Flujo del sistema: El mensaje defectuoso no puede ser procesado adecuadamente y queda detenido en el flujo de trabajo. Al carecer de mecanismos nativos de control de flujo, estado global o recuperación automática dentro de la topología Broker, el mensaje puede perderse o bloquear la cola del consumidor (Richards & Ford, 2020, cap. 14). Aunque el resto de los procesadores continúe ejecutando eventos no relacionados, la lógica de negocio asociada a dicho mensaje queda inconclusa, induciendo un estado de inconsistencia que, sin mecanismos de compensación o intervención, puede volverse permanente en ese punto del flujo.

Experiencia del usuario final: Debido a la naturaleza asíncrona de la arquitectura, el usuario recibe una respuesta inicial inmediata confirmando la recepción de la solicitud. Sin embargo, al no materializarse el efecto downstream por el fallo en segundo plano, se genera una degradación directa en la experiencia y una pérdida de confianza, ya que el sistema no refleja el resultado final ni proporciona canales explícitos de notificación o retroalimentación sobre la falla (Richards & Ford, 2020, cap. 14).

3.2. Proponga una solución arquitectónica utilizando el Workflow Event Pattern (Patrón de Eventos de Flujo de Trabajo) descrito en el Capítulo 14. Explique detalladamente cómo este patrón mitiga el problema y qué componentes adicionales (como los workflow processor y el dashboard) serían necesarios, así como los trade-offs que introduce (ej. procesamiento fuera de orden).

Para mitigar la fragilidad del manejo de errores en arquitecturas descentralizadas, Richards y Ford (2020, cap. 14) describen el Workflow Event Pattern (Patrón de Eventos de Flujo de Trabajo). Este

patrón reactivo delega la gestión de fallos a un flujo secundario sin comprometer el procesamiento del flujo principal ni introducir un orquestador global.

¿Cómo mitiga el problema?

En lugar de detener su ejecución o descartar silenciosamente el mensaje corrupto, el Event Processor redirige el evento fallido hacia un canal secundario de errores. Esto permite que el procesador principal continúe consumiendo los eventos subsiguientes, preservando la disponibilidad y el rendimiento del sistema (Richards & Ford, 2020, cap. 14). El evento retenido es procesado de forma aislada por un componente especializado que analiza el origen de la falla, ejecuta reintentos programados, aplica compensaciones de datos o escala el caso para intervención manual.

Componentes adicionales necesarios:

Workflow Processor: Componente especializado responsable de consumir los eventos desviados al canal secundario. Evalúa la causa del fallo y coordina las acciones correctivas-reparadoras sin interferir en la topología de broker del dominio principal (Richards & Ford, 2020, cap. 14).

Dashboard de Monitoreo: Panel de control visual vinculado al Workflow Processor que expone el estado de los incidentes y eventos en reparación. Facilita la supervisión por parte de operadores humanos para detectar patrones anómalos, corregir manualmente datos corruptos e inyectar nuevamente los mensajes reparados al canal principal (Richards & Ford, 2020, cap. 14).

Trade-offs que introduce:

- Procesamiento fuera de orden (Out-of-Order Processing): Al desviar un mensaje fallido a un canal secundario mientras la cola principal continúa procesando eventos posteriores, la reincorporación del evento corregido romperá la secuencia cronológica original. La consistencia eventual prima sobre el orden estricto de los eventos (Richards & Ford, 2020, cap. 14).
- Complejidad operativa: Requiere la implementación y mantenimiento de infraestructura adicional (colas de mensajes muertos, procesadores de errores dedicados y plataformas de monitoreo), incrementando la superficie del sistema (Richards & Ford, 2020, cap. 14).
- Latencia en la resolución: La resolución de errores se vuelve asíncrona y diferida en el tiempo. La materialización final del efecto del evento puede tomar minutos u horas, haciéndolo inviable para flujos de trabajo que exijan consistencia fuerte e inmediata (Richards & Ford, 2020, cap. 14).

4. El Dilema de la Granularidad y el Acoplamiento en Microservicios

Un equipo de desarrollo está migrando un sistema monolítico de gestión de recursos humanos a Microservicios. Inicialmente, crean servicios muy pequeños para cada entidad (Empleado, Departamento, Nómina, Beneficios). Pronto descubren que completar una operación simple, como contratar a un nuevo empleado, requiere una compleja coreografía de llamadas síncronas entre casi todos los servicios, lo que resulta en un alto acoplamiento y una fragilidad similar a la del monolito original.

4.1. Basado en los principios de *bounded context* y las guías de granularidad del Capítulo 17, ¿cuál fue el error fundamental en el diseño de los servicios?

El error fundamental fue caer en el antipatrón entity trap (Richards & Ford, 2020, cap. 17), o sea, tomaron cada entidad identificada e hicieron un servicio que gestiona esa entidad. Este patrón es incorrecto porque las relaciones en las bases de datos raramente corresponden a los flujos de trabajo en la aplicación.

Por el principio de bounded context (Richards & Ford, 2020, cap. 17), cada servicio encierra un dominio o capacidad de negocio, el cual incluye lo necesario para operar dentro de la aplicación.

Siguiendo las guías de granularidad (Richards & Ford, 2020, cap. 17):

Propósito: La inspiración de crear servicios muy pequeños para cada entidad no es funcionalmente cohesiva porque los servicios en sí no contribuyen capacidad alguna a la aplicación. Por tanto, no tiene propósito crear fronteras siguiendo este modelo en la arquitectura de microservicios (Richards & Ford, 2020, cap. 17).

Transacciones: Como las bases de datos de cada servicio están desacopladas, se requiere construir transacciones distribuidas que violan estas fronteras. Aunque existen patrones como Saga, estos no pueden ser una característica dominante de la arquitectura, lo cual ocurre debido a lo pequeño que son los servicios del modelo. Por tanto, para el problema de las transacciones es un error usar estas "soluciones", un indicativo de que la granularidad del servicio no es apropiada (Richards & Ford, 2020, cap. 17).

Coreografía: Usar coreografía en un modelo con varios procesos de negocio extensivos provoca que generalmente haya servicios que actúan como mediadores complejos, o sea, se está en presencia de varias instancias del patrón front controller, lo cual también es el causante del notorio alto acoplamiento. Si el conjunto de servicios ofrece un aislamiento extremo de los datos, pero requiere comunicación extensiva para funcionar, generalmente lo ideal es juntar los servicios en uno más grande para evitar estos problemas por adelantado (Richards & Ford, 2020, cap. 17).

Como consecuencia de caer en el antipatrón, en el modelo inicial se violan márgenes que indican que la estructura inicial no es apropiada para la arquitectura de microservicios.

4.2. Proponga una nueva estrategia de descomposición. ¿Cómo aplicaría los conceptos de propósito, transacciones y coreografía para redefinir los límites de los servicios y reducir el acoplamiento? Proporcione un ejemplo de cómo agruparía los servicios para la operación de çontratación".

Se debe migrar un sistema monolítico de gestión de recursos humanos a microservicios, con entidades: Empleado, Departamento, Nómina, Beneficios. La aplicación tiene que poder al menos contratar a un nuevo empleado, deducir beneficios y generar nóminas. La estrategia es iterar sobre las opciones posibles analizando su descomposición en fronteras y encontrar una, tal que su granularidad sea apropiada para el desacoplamiento en arquitectura de microservicios.

Un ejemplo a proponer es particionar la lógica de negocio en: la parte dedicada a gestionar empleado y asignar departamento, y la parte dedicada a deducir beneficios y generar nómina. Por tanto, se crearán los dos servicios: GestiónEmpleado y CálculoNóminaBeneficios.

Propósito: GestiónEmpleado permite a la aplicación administrar su fuerza laboral; entre esas tareas está contratar a un nuevo empleado. CálculoNóminaBeneficios permite deducir los beneficios que tendrá cada empleado según su contrato y cada cierto tiempo generar una nómina a partir de estos.

Coreografía: CálculoNóminaBeneficios debe ser capaz de saber cuándo se registra un nuevo empleado para deducir su pago en el próximo ciclo. Como no requiere que la comunicación sea síncrona porque la deducción no tiene que ser inmediatamente después de contratar a un empleado, se analizarán las compensaciones de los patrones de comunicación asíncrona en microservicios, para elegir el que mejor satisfaga la tarea. Se propone usar coreografía (Richards & Ford, 2020, cap. 17), basada en el patrón broker (Richards & Ford, 2020, cap. 14), para preservar el desacoplamiento. Aunque la coreografía trae consigo problemas de gestión de errores, el evento EmpleadoContratado no representa un proceso de negocio complejo y no requiere de servicios que actúen como mediadores. Por lo tanto, resulta conveniente utilizar esta estrategia

Transacciones: La granularidad del modelo propuesto no presenta problemas de transacciones distribuidas porque las transacciones locales de GestiónEmpleado y CálculoNóminaBeneficios no necesitan estar coordinadas para funcionar, o sea, el que se ejecute o no una transacción en un servicio externo no

afecta los datos del contexto dentro de los otros.

Como anteriormente se ha analizado, entre las características de la arquitectura propuesta está el bajo acoplamiento, y teniendo en cuenta las compensaciones en la coreografía se logra un sistema robusto que recibe las múltiples ventajas que posee la arquitectura de microservicios (Richards & Ford, 2020, cap. 17).

5. Comparación de Atributos de Calidad y Trade-offs

Un director de tecnología (CTO) le pide que elija un estilo arquitectónico para un nuevo sistema de procesamiento de transacciones financieras. Los requisitos clave son: 1) Consistencia de datos absoluta (ACID), 2) Capacidad de desplegar nuevas características de negocio de forma independiente y rápida, y 3) Un equipo de desarrollo con experiencia mixta.

5.1. Compare Service-Based Architecture y Microservices Architecture en función de estos tres requisitos. ¿Cuál de las dos arquitecturas es más adecuada para cumplir con el requisito de consistencia de datos y por qué?

Comparación entre Service-Based Architecture y Microservices Architecture

1) Consistencia de datos absoluta (ACID)

Service-Based Architecture utiliza un número reducido de servicios de dominio de grano grueso (típicamente entre 4 y 12) que comparten una base de datos o esquemas relacionales consolidados (Richards & Ford, 2020, cap. 13). Esto centraliza la persistencia y permite el uso nativo de transacciones ACID tradicionales sobre una base de datos relacional, garantizando la atomicidad y consistencia inmediata de los datos financieros sin ventanas de inconsistencia (Richards & Ford, 2020, cap. 13).

Por otro lado, Microservices Architecture, aplica estrictamente el principio de aislamiento de datos (Database-per-Service), lo que rompe por completo con las transacciones ACID globales (Richards & Ford, 2020, cap. 4, 17). Para procesar operaciones que cruzan múltiples servicios, el sistema se ve obligado a operar bajo un modelo de consistencia eventual (eventual consistency) y transacciones tipo BASE, requiriendo patrones complejos de coordinación asíncrona como Sagas (Richards & Ford, 2020, cap. 4, 17).

2) Capacidad de desplegar nuevas características de negocio de forma independiente y rápida

Service-Based Architecture permite que sus servicios de dominio de grano grueso se desplieguen de forma separada (Richards & Ford, 2020, cap. 13). Sin embargo, la presencia de la base de datos compartida introduce un acoplamiento a nivel de esquema (connascence of schema), lo que significa que los cambios estructurales en los datos pueden exigir coordinación y despliegues conjuntos entre los equipos (Richards & Ford, 2020, cap. 3, 13).

Mientras que, en Microservices Architecture cada microservicio representa un artefacto autónomo e independiente dotado de su propia base de datos. Esto maximiza la agilidad al permitir despliegues continuos y frecuentes sin afectar el ciclo de vida de otros servicios (Richards & Ford, 2020, cap. 17).

3) Un equipo de desarrollo con experiencia mixta

Al contar la Service-Based Architecture con un número reducido de servicios y apoyarse en patrones de desarrollo relacionales familiares, reduce drásticamente la complejidad operativa, arquitectónica y de infraestructura para equipos con perfiles diversos o mixtos (Richards & Ford, 2020, cap. 13).

Microservices Architecture, por su parte, demanda un equipo con madurez avanzada en DevOps, gestión de redes distribuidas, gobierno y patrones complejos de resiliencia frente a los fallos de red (Richards

& Ford, 2020, cap. 17).

Criterio	Service-Based Architecture (SBA)	Microservices Architecture
Consistencia de Datos (ACID)	Muy alta / Nativa	Baja / Compleja (BASE)
Despliegue Independiente	Moderado	Alto
Adaptabilidad al Equipo (Experiencia Mixta)	Excelente	Exigente

¿Cuál es más adecuada para cumplir con la consistencia absoluta y por qué?

La arquitectura más adecuada es la Service-Based Architecture (SBA). Los requerimientos financieros que exigen consistencia de datos absoluta (ACID) entran en conflicto directo con la naturaleza distribuida y de consistencia eventual propia de los microservicios (Richards & Ford, 2020, cap. 4, 17). La SBA ofrece un híbrido pragmático donde los servicios de dominio de grano grueso pueden apoyarse en una base de datos unificada, permitiendo transacciones ACID robustas y garantizando la integridad inmediata de los datos financieros sin la abrumadora complejidad operativa de los patrones distribuidos (Richards & Ford, 2020, cap. 13).

5.2. ¿Cómo se ve afectada la capacidad de despliegue independiente (requisito 2) en cada estilo? ¿Qué trade-offs debe aceptar el CTO si prioriza la consistencia sobre la agilidad de despliegue?

Impacto en la capacidad de despliegue independiente

Microservices Architecture ofrece el nivel más alto de despliegue independiente, ya que el desacoplamiento estricto del código y de las bases de datos permite que los equipos desplieguen cambios de manera totalmente autónoma, continua y sin fricciones (Richards & Ford, 2020, cap. 17).

En Service-Based Architecture, la capacidad de despliegue independiente se ve parcialmente limitada. Aunque los servicios de dominio se despliegan por separado, la presencia de una base de datos compartida introduce un acoplamiento a nivel de datos *(connascence of schema)* que requiere sincronización entre los equipos ante modificaciones del esquema (Richards & Ford, 2020, cap. 13).

Trade-offs que debe aceptar el CTO si prioriza la consistencia sobre la agilidad de despliegue:

- Reducción de la agilidad y autonomía de despliegue:
  Se sacrifica la independencia total de despliegue característica de los microservicios, requiriendo procesos de validación y pruebas integrales más rigurosos antes de liberar actualizaciones a producción debido a la base de datos compartida (Richards & Ford, 2020, cap. 13).

- Cuellos de botella en la persistencia:
  Al compartir bases de datos entre múltiples servicios de dominio de grano grueso, la escalabilidad del almacenamiento puede enfrentar mayores puntos de contención en comparación con arquitecturas donde cada servicio aísla su propia fuente de datos (Richards & Ford, 2020, cap. 13).

- Acoplamiento estructural y de datos:
  Se asume el riesgo de un mayor acoplamiento lógico a través del almacenamiento compartido (connascence of schema), lo que exige una estricta gobernanza en el diseño de las bases de datos para evitar efectos colaterales imprevistos entre los servicios (Richards & Ford, 2020, cap. 3, 13).

- Granularidad de despliegue más amplia:
  Al estar la lógica agrupada en bloques de grano grueso (habitualmente entre 4 y 12 servicios), cualquier modificación menor en una regla de negocio puede obligar a empaquetar y desplegar todo el servicio de dominio correspondiente (Richards & Ford, 2020, cap. 13).

6. El Legado de SOA y su Influencia en Arquitecturas Modernas

La Service-Oriented Architecture (SOA) con orquestación a menudo es criticada por su complejidad y por crear un cuello de botella central (el motor de orquestación). Sin embargo, algunos de sus principios, como la reutilización de servicios, siguen siendo relevantes. SOA también admite coreografía, no solo orquestación.

6.1. Analice cómo la Microservices Architecture aborda el problema de la reutilización de una manera fundamentalmente diferente. ¿Cómo maneja la reutilización de lógica de negocio (ej. validación de clientes) vs. la reutilización operativa (ej. logging, monitoreo)?

6.2. ¿En qué escenarios un arquitecto podría todavía considerar un enfoque de orquestación (como un mediador local en microservicios o un orchestration engine en SOA) a pesar de sus desventajas? Proporcione un ejemplo de un flujo de trabajo complejo que se beneficiaría de la orquestación explícita.

La Arquitectura Orientada a Servicios (SOA) es un modelo que propone el uso de servicios para facilitar la interacción e integración (Richards & Ford, 2020). Para implementarlo se crean servicios de negocio reutilizables como Check Customer Status o Create Invoice, y luego los sistemas como Marketing o Sales apuntan a esos servicios centrales y a una base de datos compartida. El objetivo y ventaja de SOA es posibilitar la reutilización, como en el ejemplo, donde se usan los mismos servicios para aplicaciones distintas. Sin embargo, esto complica la gestión, puesto que todos llaman al bus de servicios central cada vez que necesitan uno de ellos, lo que crea acoplamiento y un cuello de botella operativo. Esta es la desventaja que planea resolver la evolución de SOA hacia los microservicios.

Los microservicios abordan la reutilización de forma diferente, sin compartir un servicio de validación central y siguiendo el principio de Bounded Context (Contexto Delimitado). Cada microservicio (Pagos, Envíos, Pedidos) contiene su propia lógica de validación de clientes adaptada a su necesidad: por ejemplo, Pagos necesita validar el crédito, mientras que Envíos necesita validar la dirección. Por tanto, no se reutiliza el código, sino el concepto (Richards & Ford, 2020).

Con respecto a la reutilización operativa (logging, monitoreo), esta se realiza mediante la infraestructura: en lugar de que los microservicios implementen su propio logging, se utilizan herramientas de plataforma como Sidecars, que se encargan de las tareas transversales en un contenedor aparte sin que el código de negocio lo sepa.

Con SOA es más común el uso de un motor de orquestación (un cerebro central que coordina cada servicio en un orden o tarea específica), lo que lo convierte en un único punto de fallo y un riesgo para la disponibilidad, aunque a veces también permite coreografía, la cual es preferida en los microservicios. Ella consiste en un manejo por eventos: por ejemplo, "se creó un pedido", y los demás servicios reaccionan a ese evento por su cuenta.

Pero aun así, en los microservicios en ocasiones debe considerarse la orquestación. Esto ocurre cuando existe un flujo de trabajo complejo con pasos secuenciales estrictos, condiciones lógicas, o cuando hay necesidad de hacer un seguimiento paso a paso de cada operación para deshacerlas o supervisarlas. Esto se puede lograr con un mediador local (un orquestador de menor escala).

Un ejemplo de escenario donde es útil es el proceso de reserva de un viaje que incluya vuelo, hotel y alquiler de coche. Con orquestación, el flujo sería: reservar el vuelo; si este se reserva, reservar el hotel, y si el hotel se reserva, entonces también el coche. Luego, si el coche falla, se cancela el hotel, y si el hotel se cancela, se cancela el vuelo, puesto que la solicitud de reservar los tres no se completó y ahora no se quiere reservar ninguno. Si esto se hubiera hecho con coreografía pura, no se sabría qué cancelación requiere de otra, porque no se tendría claro qué paso implicó a cuál o en qué orden se realizaron las operaciones. Gracias al orquestador, que conoce el flujo de eventos, pueden rastrearse los pasos hacia atrás para saber cómo revertir las operaciones (Patrón Saga y Transacciones Compensatorias, Ford et al., 2021). El orquestador logra la centralización de la lógica, garantizando la consistencia del estado del flujo sin la complejidad de coordinar eventos descentralizados, para que un solo fallo no deje el sistema en un estado inconsistente.

7. Declaración de Uso de la Inteligencia Artificial

Para la elaboración de este seminario, el equipo utilizó la herramienta Gemini (Google) con el propósito de revisar la estructura y estilo del documento y la ortografía del mismo, así como para la creación de las tablas que aquí aparecen. Todo el contenido generado fue revisado, verificado y adaptado por el equipo, que asume la responsabilidad final del documento.

8. Referencias Bibliográficas

- Ford, N., Richards, M., Sadalage, P., & Dehghani, Z. (2021). *Software Architecture: The Hard Parts: Distributed Data Concepts and Decision Making.* O'Reilly Media.
- Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD Record*, 16(3), 249–259.
- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach.* O'Reilly Media.